

Introduce *Cpp17Swappable* as additional convenience requirements

- [Introduction](#)
- [Revision History](#)
- [Discussion](#)
- [Rationale](#)
- [Proposed Resolution](#)
- [Acknowledgements](#)
- [Bibliography](#)

Introduction

This proposal suggests to introduce an additional requirements set to simplify, clarify, and harmonize various swappable specifications in the Standard Library.

Revision History

- Following LWG review recommendations, *Cpp17Swappable* requirements have been withdrawn from `unique_ptr::delete_type`, `pair`, and `tuple` swaps.

Discussion

[N3048](#) introduced requirements sets for various swappable requirements because at that point we had eradicated the initial C++11 concepts from the working draft and had no corresponding substitute by the core language to express them. Nonetheless they had been introduced attempting to get similar specification advantages. At that time, we introduced specific swappable requirements for those situations that seemed most prominently occurring in the working draft, namely:

1. An rvalue or lvalue `t` is *swappable* if and only if `t` is swappable with any rvalue or lvalue, respectively, of type `T`.
2. A type `X` meeting any of the iterator requirements (25.3 [iterator.requirements]) meets the *Cpp17ValueSwappable* requirements if, for any dereferenceable object `x` of type `X`, `*x` is swappable.

At that time it seemed especially useful to introduce the term *Cpp17ValueSwappable* because the specification of various algorithms could be considerably simplified that way.

Rationale

The [N3048](#) paper did attempt to keep the changes to the working draft rather small, but with its future evolvement one could say that we underestimated the special case of "swappable lvalues", because we didn't introduce a specific name for them. The effect of the current situation is that various places in the specification of types uses a mixture of requirement set names such as *Cpp17MoveAssignable* and *Cpp17MoveConstructible* together with "lvalues of type `X` are swappable", leading to unnecessary paragraph splitting and often unnecessary long wording.

This paper attempts to improve the specification state by introducing a new named requirement set, *Cpp17Swappable*, as a new shortcut when we attempt to specify swappable lvalues of a specific (homogeneous) type and to use this new shortcut throughout the library specification. In most cases, this leads to a shorter and clearer specification.

In an earlier draft version of this proposal we decided to apply this new set to additional places ([unique.ptr.single.modifiers], [pairs.pair], [tuple.swap]) where the involved types could have reference types. But feedback from the LWG review group showed that requirement sets are not well-suited here because we cannot simply apply reference collapsing rules as we would do in templates. This basically means that we are affected by the long existing LWG issue [LWG2146](#) here. An alternative solution could have been to use `remove_reference_t<Type>` to cope for the reference cases, but that seemed less appealing because it would in some places make the requirement too general, which was not wanted. In all such situations we have therefore kept the preexisting style or improved the current wording for other reasons.

It should be emphasized that the intention of this paper is just a simplification of existing specification, but *not* a change of semantics. In theory, this request could have been handled as a plain editorial change request, but the author thinks that the amount of wording changes is sufficiently high to approve them like an ordinary proposal.

Proposed resolution

The proposed wording changes refer to [N4917](#).

1. Modify in 16.4.4.3 [swappable.requirements] Swappable requirements as indicated:

-4- An rvalue or lvalue `t` is *swappable* if and only if `t` is swappable with any rvalue or lvalue, respectively, of type `T`.

~~?- A type `X` meets the `Cpp17Swappable` requirements if lvalues of type `X` are swappable.~~

-5- A type `X` meeting any of the iterator requirements (25.3 [iterator.requirements]) meets the `Cpp17ValueSwappable` requirements if, for any dereferenceable object `x` of type `X`, `*x` is swappable.

-6- [Example 1: User code can ensure that the evaluation of swap calls is performed in an appropriate context under the various conditions as follows:

```
#include <cassert>
#include <utility>

// Preconditions: std::forward<T>(t) is swappable with std::forward<U>(u).
template<class T, class U>
void value_swap(T&& t, U&& u) {
    using std::swap;
    swap(std::forward<T>(t), std::forward<U>(u)); // OK, uses "swappable with" conditions
                                                // for rvalues and lvalues
}

// Preconditions: lvalues of T meets the Cpp17Swappable requirementsare swappable.
template<class T>
void lv_swap(T& t1, T& t2) {
    using std::swap;
    swap(t1, t2); // OK, uses swappable conditions for lvalues of type T
}

[...]
```

2. Modify 16.4.4.4 [nullablepointer.requirements] `Cpp17NullablePointer` requirements as indicated:

-1- A `Cpp17NullablePointer` type is a pointer-like type that supports null values. A type `P` meets the `Cpp17NullablePointer` requirements if:

(1.1) — `P` meets the `Cpp17EqualityComparable`, `Cpp17DefaultConstructible`, `Cpp17CopyConstructible`, `Cpp17CopyAssignable`, `Cpp17Swappable`, and `Cpp17Destructible` requirements,

~~(1.2) — lvalues of type `P` are swappable (16.4.4.3 [swappable.requirements]);~~

(1.3) — [...]

(1.4) — [...]

3. Modify 16.4.4.6.1 [allocator.requirements.general] as indicated:

typename X::propagate_on_container_swap

-86- *Result*: Identical to or derived from true_type or false_type.

-87- *Returns*: true_type only if an allocator of type X should be swapped when the client container is swapped; if so, values of type X shall meet the Cpp17Swappable requirements (16.4.4.3 [swappable.requirements]) and the swap operation shall not throw exceptions.

4. Modify 17.9.2.1 [support.srcloc.class.general] as indicated:

-1- The type source_location meets the Cpp17DefaultConstructible, Cpp17CopyConstructible, Cpp17CopyAssignable, Cpp17Swappable, and Cpp17Destructible requirements (16.4.4.2 [utility.arg.requirements], 16.4.4.3 [swappable.requirements]). Values of type source_location are swappable (16.4.4.3 [swappable.requirements]). All of the following conditions are true:

5. Keep 20.3.1.3.6 [unique.ptr.single.modifiers] unchanged:

[Drafting note: deleter_type is allowed to be of reference type, therefore we don't use Cpp17Swappable here.]

constexpr void swap(unique_ptr& u) noexcept;

-6- *Preconditions*: get_deleter() is swappable (16.4.4.3 [swappable.requirements]) and does not throw an exception under swap.

-7- *Effects*: Invokes swap on the stored pointers and on the stored deleters of *this and u.

6. Keep 22.3.2 [pairs.pair] unchanged:

[Drafting note: Both T1 and T2 are allowed to be of reference type, therefore we don't use Cpp17Swappable here.]

constexpr void swap(pair& p) noexcept(see below);
constexpr void swap(const pair& p) const noexcept(see below);

-51- *Mandates*:

(51.1) — For the first overload, is_swappable_v<T1> is true and is_swappable_v<T2> is true.

(51.2) — For the second overload, is_swappable_v<const T1> is true and is_swappable_v<const T2> is true.

-52- *Preconditions*: first is swappable with (16.4.4.3 [swappable.requirements]) p.first and second is swappable with p.second.

-53- *Effects*: Swaps first with p.first and second with p.second.

-54- *Remarks*: [...]

7. Modify 22.4.4.3 [tuple.swap] as indicated:

[Drafting note: Any of the types in `Types` are allowed to be of reference type, therefore we don't use `Cpp17Swappable` here. But feedback from the review group improves the existing specification to clarify that we always apply swap on lvalues due to the direct usage of `*this` and `rhs` as arguments of `get`. Another side-effect of the new wording is the ensured ordering of element-wise swap which corresponds to the ensured ordering of assignments in all assignment operations. This clarification shouldn't have impact on existing implementations, which all seem to use the element-wise order already.]

```
constexpr void swap(tuple& rhs) noexcept(see below);  
constexpr void swap(const tuple& rhs) const noexcept(see below);
```

-?- Let i be in the range $[0, \text{sizeof} \dots (\text{Types}))$ in order.

-1- *Mandates*:

(1.1) — For the first overload, `(is_swappable_v<Types> && ...)` is true.

(1.2) — For the second overload, `(is_swappable_v<const Types> && ...)` is true.

-2- *Preconditions*: For all i , `get< $i$ >(*this)` is swappable with (16.4.4.3 [swappable.requirements]) `get< $i$ >(rhs)` the corresponding element in `rhs`.

-3- *Effects*: For each i , calls swap for `get< $i$ >(*this)` element in `*this` and with `get< $i$ >(rhs)` its corresponding element in `rhs`.

-4- *Throws*: Nothing unless one of the element-wise swap calls throws an exception.

-5- *Remarks*: The exception specification is equivalent to [...]

8. Modify 22.5.3.5 [optional.swap] as indicated:

```
constexpr void swap(optional& rhs) noexcept(see below);
```

-1- *Mandates*: `is_move_constructible_v<T>` is true.

-2- *Preconditions*: T meets the `Cpp17Swappable` requirements Lvalues of type T are swappable.

9. Modify 22.6.3.7 [variant.swap] as indicated:

```
constexpr void swap(variant& rhs) noexcept(see below);
```

-1- *Mandates*: `is_move_constructible_v< $T_i$ >` is true for all i .

-2- *Preconditions*: Each T_i meets the `Cpp17Swappable` requirements Lvalues of type T_i are swappable (16.4.4.3 [swappable.requirements]).

10. Modify 22.10.19 [unord.hash] as indicated:

-5- An enabled specialization `hash<Key>` will:

(5.1) — meet the `Cpp17Hash` requirements (Table 38 [tab:cpp17.hash]), with `Key` as the function call argument type, the `Cpp17DefaultConstructible` requirements (Table 31 [tab:cpp17.defaultconstructible]), the `Cpp17CopyAssignable` requirements (Table 35 [tab:cpp17.copyassignable]), the `Cpp17Swappable` requirements (16.4.4.3 [swappable.requirements]),

(5.2) — be swappable (16.4.4.3 [swappable.requirements]) for lvalues,

(5.3) — [...]

(5.4) — [...]

11. Modify 22.14.6.1 [formatter.requirements] as indicated:

-1- A type *F* meets the *BasicFormatter* requirements if:

(1.1) — it meets the

(1.1.1) — *Cpp17DefaultConstructible* (Table 31),

(1.1.2) — *Cpp17CopyConstructible* (Table 33),

(1.1.3) — *Cpp17CopyAssignable* (Table 35), and

(1.1.?) — *Cpp17Swappable* (16.4.4.3 [swappable.requirements]), and

(1.1.4) — *Cpp17Destructible* (Table 36)

requirements, and

(1.2) — it is swappable (16.4.4.3 [swappable.requirements]) for lvalues, and

(1.3) — the expressions shown in Table 74 are valid and have the indicated semantics.

12. Modify 24.2.2.2 [container.reqmts] as indicated:

[Drafting note: The suggested changes for p37 and p41 provide an alternative resolution for [editorial issue 5627](#)]

`a.insert(p, i, j)`

-36- *Result*: iterator.

-37- *Preconditions*: *T* is *Cpp17EmplaceConstructible* into *x* from **i*. For vector and deque, *T* is also *Cpp17MoveInsertable* into *x*, and *T* meets the *Cpp17MoveConstructible*, *Cpp17MoveAssignable*, and *Cpp17Swappable* (16.4.4.3 [swappable.requirements]) *requirements*. Neither *i* nor *j* are iterators into *a*.

-38- [...]

-39- [...]

`a.insert_range(p, rg)`

-40- *Result*: iterator.

-41- *Preconditions*: *T* is *Cpp17EmplaceConstructible* into *x* from **ranges::begin(rg)*. For vector and deque, *T* is also *Cpp17MoveInsertable* into *x*, and *T* meets the *Cpp17MoveConstructible*, *Cpp17MoveAssignable*, and *Cpp17Swappable* (16.4.4.3 [swappable.requirements]) *requirements*. *rg* and *a* do not overlap..

-42- [...]

-43- [...]

[...]

-65- The expression `a.swap(b)`, for containers `a` and `b` of a standard container type other than array, shall exchange the values of `a` and `b` without invoking any move, copy, or swap operations on the individual container elements. ~~Lvalues of a~~ Any Compare, Pred, or Hash types belonging to `a` and `b` shall ~~meet the Cpp17Swappable requirements~~ be swappable and shall be exchanged by calling `swap` as described in 16.4.4.3 [swappable.requirements]. If `allocator_traits<allocator_type>::propagate_on_container_swap::value` is true, then ~~lvalues of type~~ `allocator_type` shall ~~meet the Cpp17Swappable requirements~~ be swappable and the allocators of `a` and `b` shall also be exchanged by calling `swap` as described in 16.4.4.3 [swappable.requirements].

13. Modify 25.3.5.2 [iterator.iterators] as indicated:

-2- A type `x` meets the *Cpp17Iterator* requirements if:

(2.1) — `x` meets the *Cpp17CopyConstructible*, *Cpp17CopyAssignable*, *Cpp17Swappable*, and *Cpp17Destructible* requirements (16.4.4.2 [utility.arg.requirements], 16.4.4.3 [swappable.requirements]) ~~and lvalues of type `x` are swappable (16.4.4.3 [swappable.requirements])~~, and

(2.2) — [...]

(2.3) — [...]

14. Modify 29.3 [time.clock.req] as indicated:

-4- A type `TC` meets the *Cpp17TrivialClock* requirements if:

(4.1) — `TC` meets the *Cpp17Clock* requirements,

(4.2) — the types `TC::rep`, `TC::duration`, and `TC::time_point` meet the *Cpp17EqualityComparable* (Table 29) and *Cpp17LessThanComparable* (Table 30) and *Cpp17Swappable* (16.4.4.3 [swappable.requirements]) requirements and the requirements of numeric types (28.2 [numeric.requirements]). [Note 3: This means, in particular, that operations on these types will not throw exceptions. — end note]

~~(4.3) — lvalues of the types `TC::rep`, `TC::duration`, and `TC::time_point` are swappable (16.4.4.3 [swappable.requirements])~~,

(4.4) — the function `TC::now()` does not throw exceptions, and

(4.5) — the type `TC::time_point::clock` meets the *Cpp17TrivialClock* requirements, recursively.

Acknowledgements

Thanks to the [LWG group that reviewed](#) this proposal and provided helpful improvement suggestions.

Bibliography

[N4917] Thomas Köppe: "Working Draft, Standard for Programming Language C++", 2022

<https://wg21.link/n4917>

[N3048] Daniel Krügler, Mike Spertus, Stefanus Du Toit, Walter E. Brown: "Defining Swappable Requirements", 2010

<https://wg21.link/n3048>

[LWG2146] Nikolay Ivchenkov: "Are reference types Copy/Move-Constructible/Assignable or Destructible?", 2012

<https://wg21.link/lwg2146>